Representação distribuída de texto

Ivandré Paraboni
EACH - USP

Créditos

Material adaptado do original em inglês:
Dan Jurafsky and James H. Martin
“Speech and Language Processing”
https://web.stanford.edu/~jurafsky/slp3/ (3rd edition, draft)

Com acréscimos de

- material didático cedido pelo prof. Thiago Pardo (NILC-ICMC)
- Documentação Scikit-learn em https://scikit-learn.org/stable/

Visão geral

Exercício anterior: classificação de gênero no córpus BRmoral
Representação distribuída de palavras (*word embeddings*)
Viés de representação
Word embeddings na prática

Exercício anterior: classificação de gênero no córpus BRmoral

Resultados até aqui

- Classificação de gênero usando *Split* de treinamento/teste definido
- Medida F1 macro
- Todos modelos baseados em palavras (word)

- Baseline classe majoritária : 0.40
- LogReg usando BoW : 0.63
- LogReg usando TF-IDF : 0.65
- LogReg usando TF-IDF kbest=3000 : 0.62

Resultados até aqui

- Classificação de gênero usando *Split* de treinamento/teste definido
- Medida F1 macro
- Todos modelos baseados em palavras (word)

- Baseline classe majoritária : 0.40
- LogReg usando BoW : 0.63
- LogReg usando TF-IDF : 0.65
- LogReg usando TF-IDF kbest=3000 : 0.62
- LogReg usando TF-IDF kbest=9000 : 0.70

Considerações

- **Menos é mais**: prefira modelos simples, com poucas características, e vá aumentando gradativamente

- Se o volume de instâncias for muito grande, considere trabalhar com apenas uma porção dos dados, e somente depois de estar satisfeito com o modelo faça o teste completo

- Serão necessários muitos experimentos e muita otimização até chegar ao modelo ideal.

O que mais poderia ser melhorado?

- **Otimização** de parâmetros: todo algoritmo – inclusive LogisticRegression, possui diversos parâmetros a serem configurados

- Outros **algoritmos** de AM: scikit-learn possui dezenas deles!
  SGD, MLP...

- Outros métodos de **seleção de atributos** / redução de dimensionalidade
  (especialmente PCA)

- Outras **representações** textuais (n-gramas de palavras ou caracteres)

- **Representação distribuída** de palavras ➔ embeddings

Representação distribuída de palavras (*word embeddings*)

Problemas com o modelo BoW

Um texto é representado na forma de um vetor de **grande dimensionalidade**

Da ordem de várias de zenas de milhares de características

A maioria das posições possui valor zero (**esparsidade**)

Palavras semanticamente muito próximas **não têm relação entre si**, e.g., “vendido” e “vendida”, “gostar” e “preferir”, “aluno” e “*alluno*” etc.

Ideia: agrupar palavras de acordo com o uso (contextual) em uma representação densa (quase todas posições <> 0) e de baixa dimensionalidade (+/- 300)

Contexto

- O contexto de uma palavra w é o conjunto de palavras anteriores e posteriores a ela, em uma janela de tamanho pré-definido

w-3 w-2 w-1 w w+1 w+2 w+3

E por que usar vetores densos?

- Vetores menores são mais eficientes
- Permitem obter melhores generalizações
- Permitem capturar sinonímia (e.g., “carro” e “automóvel”)
- Na prática tendem a funcionar melhor

Representação distribuída

- Ideia geral: duas palavras que aparecem nos mesmos contextos têm significado semelhante

    - Ele está dirigindo um {carro, automóvel, Honda Civic}

Representação distribuída

- Ideia geral: duas palavras que aparecem nos mesmos contextos têm significado semelhante

  - Ele está dirigindo um {carro, automóvel, Honda Civic}
  - Ela está tomando {água, vinho, suco, leite ...

Representação distribuída

- Ideia geral: duas palavras que aparecem nos mesmos contextos têm significado semelhante

  - Ele está dirigindo um {carro, automóvel, Honda Civic}
  - Ela está tomando {água, vinho, suco, leite ... comprimidos, cuidado}

Representação distribuída

- Ideia geral: duas palavras que aparecem nos mesmos contextos têm significado semelhante
    - Ele está dirigindo um {carro, automóvel, Honda Civic}
    - Ela está tomando {água, vinho, suco, leite ... comprimidos, jeito}
- Palavras com mesmo significado podem ser tratadas como se fossem (em certa medida) uma só
    - Redução do modelo

Word embeddings

- Cada palavra é representada por um vetor de valores reais de dimensão fixa e pré-estabelecida (significado está “embutido” no vetor)
    - e.g., a palavra “cachorro” é representada por um vetor de 300 posições etc.

Word embeddings

- Cada palavra é representada por um vetor de valores reais de dimensão fixa e pré-estabelecida (significado está “embutido” no vetor)
    - e.g., a palavra “cachorro” é representada por um vetor de 300 posições etc.
- O vetor representa o significado da palavra em um nível de detalhe tal que palavras semanticamente próximas possuem vetores próximos
    - Distância entre dois vetores = álgebra

Word embeddings

- Cada palavra é representada por um vetor de valores reais de dimensão fixa e pré-estabelecida (significado está “embutido” no vetor)
    - e.g., a palavra “cachorro” é representada por um vetor de 300 posições etc.
- O vetor representa o significado da palavra em um nível de detalhe tal que palavras semanticamente próximas possuem vetores próximos
    - Distância entre dois vetores = álgebra
- Assim, “cachorro” é próximo de “gato”, mais próximo ainda de “cão”, porém bem distante de “computador”

(pense no contexto em que cada uma seria utilizada)

Word embeddings

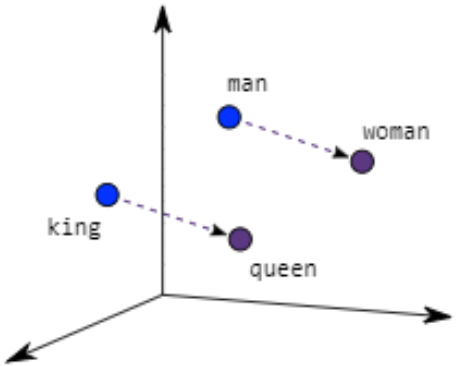
- Intuição: o significado de uma palavra é determinado pelo seu uso
- Ou, mais especificamente, pelo **contexto** no qual aparece
  - Palavras anteriores e posteriores a ela
- Ela gosta de comer _W_ depois do almoço.
  - O que seria um _W_ provável neste contexto?

Word embeddings

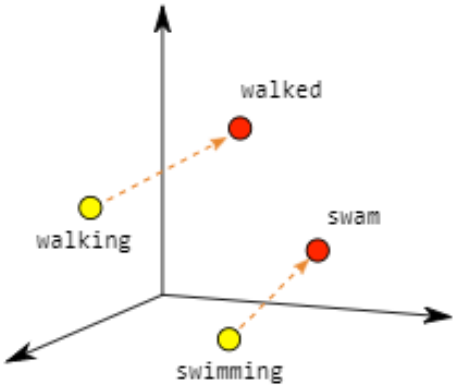
- Ela gosta de comer _W_ depois do almoço.

- O que seria um _W_ provável neste contexto?
  - Sobremesa?
  - Pudim
  - Sorvete
  - etc...
- Embora não sejam sinônimos, estas palavras são semanticamente próximas
- E assim possuem representações relativamente próximas

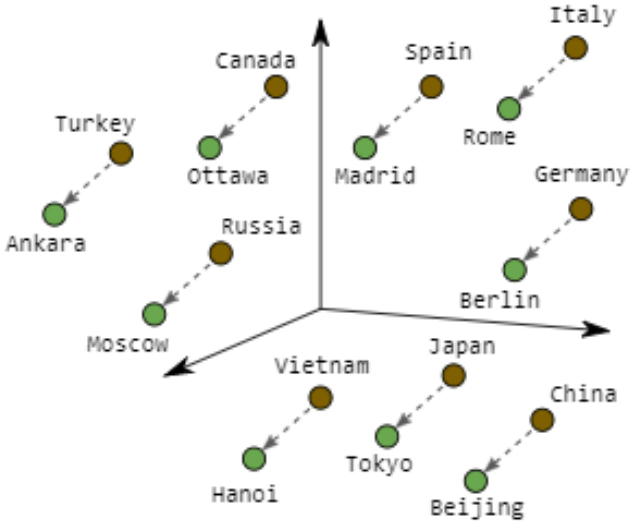
Relações no espaço vetorial



Male-Female



Verb Tense



Country-Capital

https://developers.google.com/machine-learning/crash-course/embeddings/translating-to-a-lower-dimensional-space

Álgebra de vetores de embeddings (Mikolov et. al., 2013)

Exemplos famosos

- vector(‘king’) - vector(‘man’) + vector(‘woman’) ≈ vector(‘queen’)
- vector(‘Paris’) - vector(‘France’) + vector(‘Italy’) ≈ vector(‘Rome’)

Similaridade semântica na prática

Os conceitos “água” e “vinho” ocorrem em muitos contextos semelhantes

- Tomar X
- Um copo de X

Similaridade semântica na prática

Os conceitos “água” e “vinho” ocorrem em muitos contextos semelhantes

Tomar X

Um copo de X

Em menor grau, “água” e “sopa” também

Tomar X

Um copo de (X=água) ou um prato de (X=sopa)

Similaridade semântica na prática

Os conceitos “água” e “vinho” ocorrem em muitos contextos semelhantes

Tomar X

Um copo de X

Em menor grau, “água” e “sopa” também

Tomar X

Um copo de (X=água) ou um prato de (X=sopa)

A similaridade entre “água” e “mar” também existe, mas em outra dimensão

Tomar (X=água)

O (X=mar) está agitado / A (X=água) está agitada

Erros ortográficos, neologismos, flexões etc.

- Qual a posição de *”**sobremeza**” no espaço vetorial?

Erros ortográficos, neologismos, flexões etc.

- Qual a posição de *”**sobremeza**” no espaço vetorial?
- E como “**zuado**” seria representado?

Erros ortográficos, neologismos, flexões etc.

- Qual a posição de *”**sobremeza**” no espaço vetorial?
- E como “**zuado**” seria representado?

- O significado de uma palavra é dado pelo seu contexto.

Erros ortográficos, neologismos, flexões etc.

- Qual a posição de *”**sobremeza**” no espaço vetorial?
- E como “**zuado**” seria representado?

- O significado de uma palavra é dado pelo seu contexto.

- A grafia incorreta de “sobremesa” é usada nas mesmas situações que a grafia correta = vetores muito próximos

Erros ortográficos, neologismos, flexões etc.

- Qual a posição de *”**sobremeza**” no espaço vetorial?
- E como “**zuado**” seria representado?

- O significado de uma palavra é dado pelo seu contexto.

- A grafia incorreta de “sobremesa” é usada nas mesmas situações que a grafia correta = vetores muito próximos
- O neologismo “zuado” aparece na posição típica de palavras de conotação negativa, e.g., “ruim”, “bagunçado” etc.

Erros ortográficos, neologismos, flexões etc.

- Qual a posição de *”**sobremeza**” no espaço vetorial?
- E como “**zuado**” seria representado?

- O significado de uma palavra é dado pelo seu contexto.

- A grafia incorreta de “sobremesa” é usada nas mesmas situações que a grafia correta = vetores muito próximos
- O neologismo “zuado” aparece na posição típica de palavras de conotação negativa, e.g., “ruim”, “bagunçado” etc.
- Flexões (como de gênero) são vistas como um termo único (cansado/a)

Erros ortográficos, neologismos, flexões etc.

- Qual a posição de *”**sobremeza**” no espaço vetorial?
- E como “**zuado**” seria representado?

- O significado de uma palavra é dado pelo seu contexto.

- A grafia incorreta de “sobremesa” é usada nas mesmas situações que a grafia correta = vetores muito próximos
- O neologismo “zuado” aparece na posição típica de palavras de conotação negativa, e.g., “ruim”, “bagunçado” etc.
- Flexões (como de gênero) são vistas como um termo único (cansado/a)
- *Embeddings* tratam de forma natural e elegante todos estes fenômenos
    - Desde que haja um número suficiente de exemplos no treinamento

Dimensões do vetor de *embeddings*

- Podem ser vistas como características semânticas do conceito (palavra) a que se referem
    - e.g., cor, tamanho, procedência etc.
- Mas sem interpretação humana
- Mais dimensões = mais informação sobre o conceito
    - Mas só faz sentido usar vetores maiores se houver dados para preenchê-lo

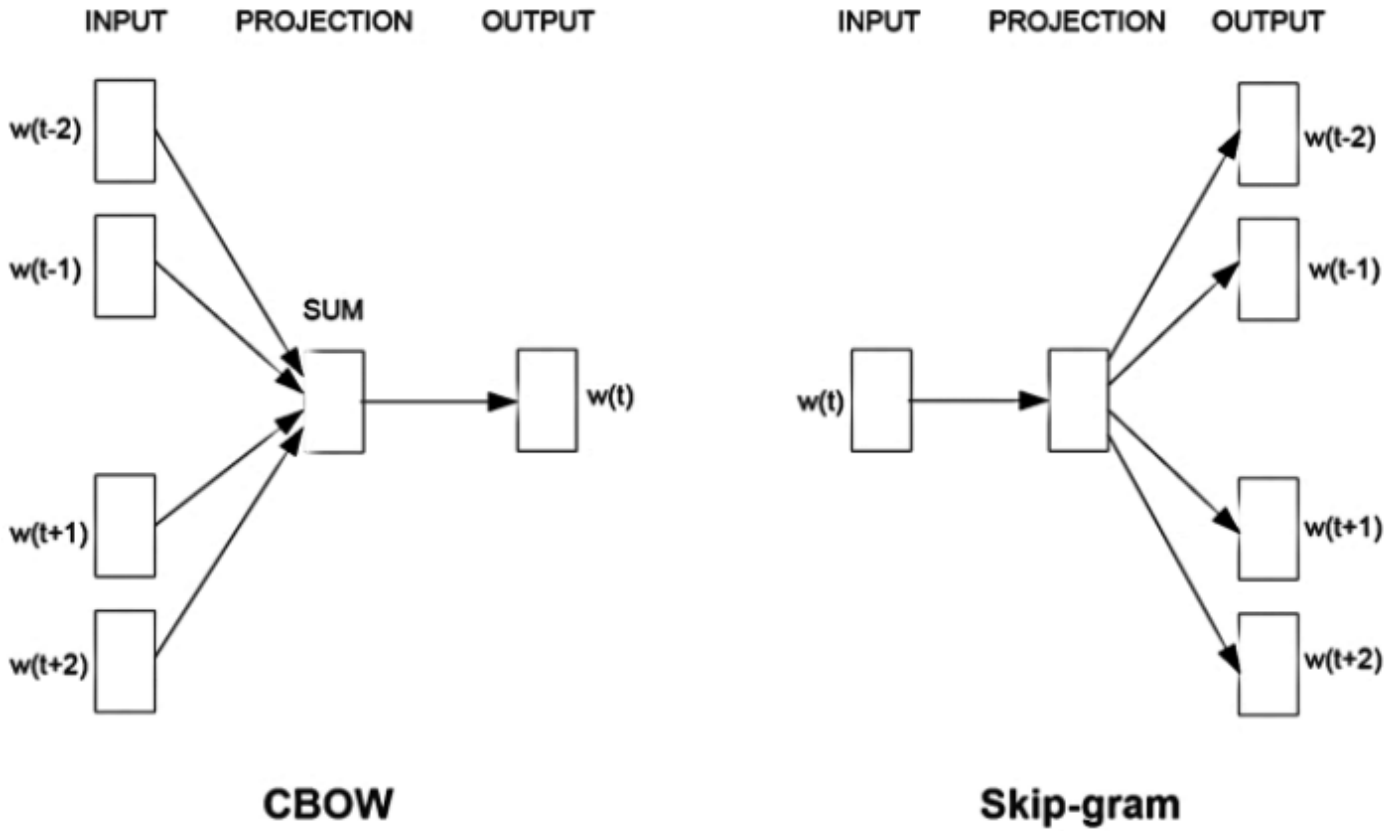
Quantas dimensões usar?

- Tamanho usual, para representações estáticas de palavras, é em torno de 300 dimensões
  - Dificilmente acima disso
  - Para córpus pequenos, 100 dimensões pode ser o suficiente
  - Mas tipicamente não menos do que 50 – tende a perder propriedades importantes

word2vec

- Apresentado em Mikolov (2013)
- Cômputo de embeddings **estáticos** (ou de contexto fixo)
- Implementação eficiente de 2 métodos de treinamento de *word embeddings*:
  - Continuous bag-of-words (CBOW)
  - Skip-gram
- Pouca ou nenhuma diferença do ponto de vista da aplicação

CBOW e SKIP-GRAM (Mikolov, 2013)



Modelos de *word embeddings*

- Cada palavra é um vetor de N dimensões
- Amplamente utilizados na construção de modelos textuais, especialmente em combinação com redes neurais
- Podem ser treinados a partir do próprio conjunto de dados (se for suficientemente grande) ou obtidos de repositórios de embeddings pré-treinados
- Em Python, a classe word2vec é integrante da biblioteca Gensim
- Keras etc. possuem implementações próprias de uma camada de *embeddings* nos modelos neurais oferecidos

Pré-processamento versus *embeddings*

Um problema de compatibilidade

- Métodos tradicionais de pré-processamento simplificam o texto de entrada
  - Remoção de *stopwords* (eu, para, etc.)
  - *Stemming* (casariam, casinhas => cas)
  - Lematização (meninas,menininho => menino)
- Estas simplificações podem causar (ligeira?) mudança de significado.
  - **Original**: Eu queria ir para casa.
  - **Sem *stopwords* e lematizado**: querer ir casa

Além da mudança de significado

- A alteração de significado pode ser aceitável diante dos benefícios do pré-processamento
    - Maior rapidez de treino e maior acurácia
- No entanto, este tipo de tratamento resulta em textos atípicos, e possivelmente não-gramaticais
    - **Sem *stopwords* e lematizado**: querer ir casa

Um problema para modelos de *embeddings*

- Modelos de *embeddings* normalmente não tiveram acesso a textos atípicos em tempo de treinamento.
  - Sequências inválidas “ir casa”
  - Palavras inexistentes “cas”
- A remoção de *stopwords*, lematização e *stemming* não são técnicas recomendáveis quando utilizando *embeddings*.
  - A menos que o próprio modelo de *embeddings* tenha sido treinado com base em textos modificados da mesma forma.

Prática

Duas questões

- Como computar estas representações?
- E como usá-las em classificação de textos?

Computando *word embeddings* com word2vec

word2vec e scikit-learn

- Implementação popular disponibilizada pela biblioteca **gensim**
- Podem ser pré-treinados ou construídos a partir do córpus existente

  (treinamento + teste)

Embeddings pré- versus auto-treinados

- Use embeddings pré-treinados quando o volume de dados disponíveis for pequeno
    - Caso contrário, embeddings auto-treinados tendem a ser melhores
    - Mas tome cuidado com a tokenização!
- Para gêneros textuais (muito) diferentes da língua padrão, auto treinamento pode apresentar melhores resultados
    - e.g., textos provenientes de redes sociais etc.

Word embeddings auto-treinados

- Importante: texto de treinamento precisa ser tokenizado da mesma forma que os dados textuais da aplicação – entrada é uma lista de tokens
    - Utilizando o mesmo *analyzer* em ambos os casos

```
# textcol é o nome da coluna de texto no dataframe
text_column = df.loc[:,textcol].str.lower()
text_column = text_column.values   # ou to_numpy() – veja documentação pandas
vect = TfidfVectorizer(ngram_range=[1,1],analyzer='word')
X = vect.fit_transform(df[textcol])   # para uso posterior, na classificação

meuAnalyzer = vect.build_analyzer()
tokenList = []
for item in text_column:
        tokens = meuAnalyzer(item)
        tokenList.append(tokens)
```

Word embeddings auto-treinados

```
from gensim.models import Word2Vec

tamanho = 300
janela = 1
minimoC = 1

model = Word2Vec(tokenList, size=tamanho, window=janela, min_count=minimoC, sg=1, seed=10)

model.wv.save("meusEmbeddings300.kv")
```

Word embeddings auto-treinados

```
from gensim.models import Word2Vec

tamanho = 300
janela = 1
minimoC = 1

model = Word2Vec(tokenList, size=tamanho, window=janela, min_count=minimoC, sg=1, seed=10)

model.wv.save("meusEmbeddings300.kv")
```

Uso:

```
from gensim.models import KeyedVectors

model = KeyedVectors.load("meusEmbeddings300.kv", mmap='r')
```

Usando *word embeddings* na classificação de texto

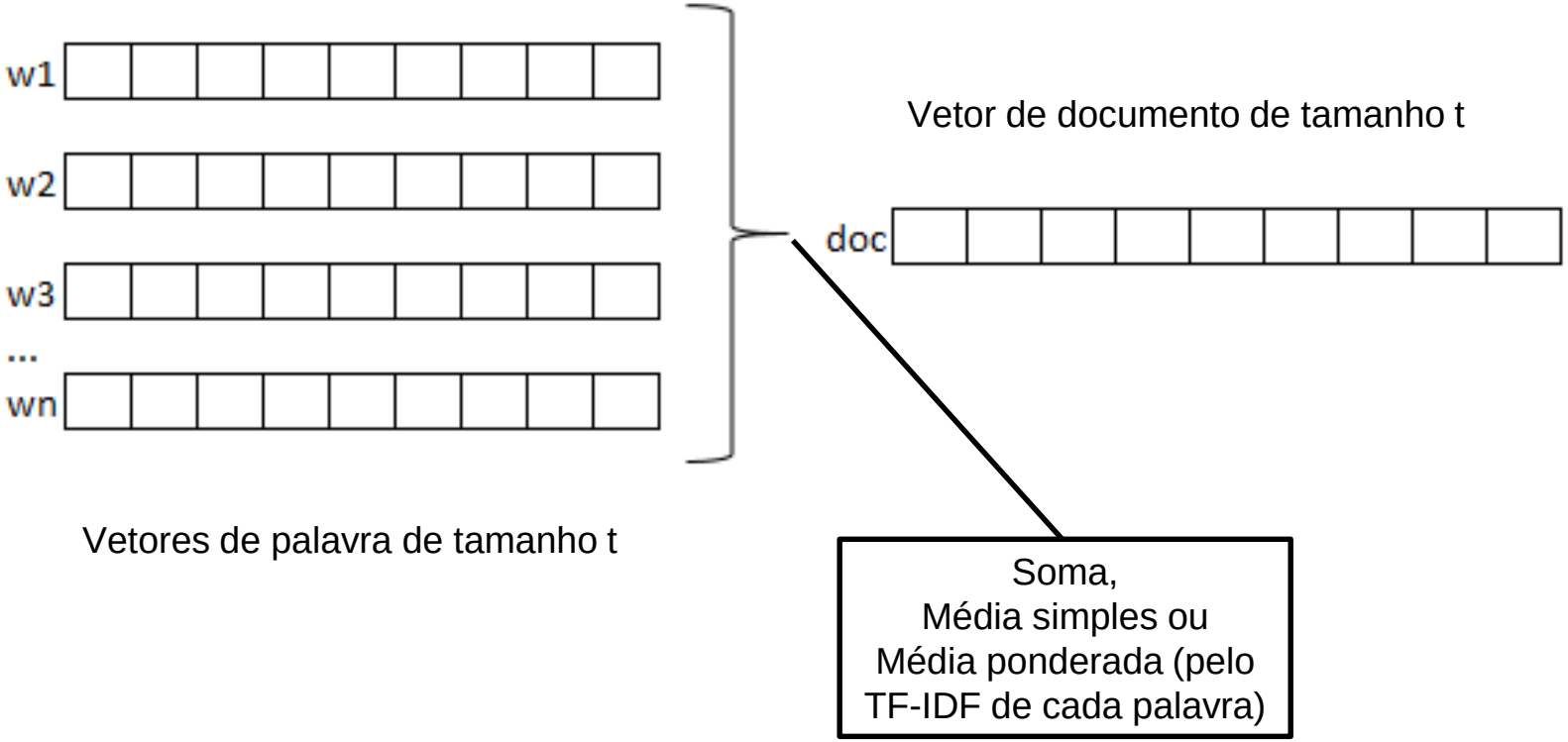
Como representar um texto?

- *Word embeddings* representam palavras individuais
- Mas normalmente não queremos classificar palavras individuais, e sim classificar conjuntos de várias palavras
    - uma frase
    - um documento
    - um *tweet*
    - etc.
- Necessidade de combinar *word embeddings* individuais de alguma forma

Alternativa 1: média ou soma de *embeddings*

- Estratégia: combinar todos os embeddings de palavras em um único vetor
  - Soma
  - Média simples
  - Média ponderada (e.g., por TF-IDF)
- Vetor resultante possui a mesma dimensão dos vetores de palavras
- Simplificação brutal da realidade...
  - Mas que muitas vezes funciona!

Média / soma de *embeddings* de palavras w1..wn



Documento representado pela média de vetores

```
import numpy as np

def avg_document_vector(word2vec_model, doc):
    doc = [word for word in doc if word in word2vec_model.key_to_index]
    if doc:
        return np.mean(word2vec_model[doc], axis=0)
    else:
        return np.zeros(word2vec_model.vector_size)
```

Documento representado pela média de vetores

```
import numpy as np

def avg_document_vector(word2vec_model, doc):
    doc = [word for word in doc if word in word2vec_model.key_to_index]
    if doc:
        return np.mean(word2vec_model[doc], axis=0)
    else:
        return np.zeros(word2vec_model.vector_size)
```

Exercício: e a média ponderada por TF-IDF???

Considerações

- Ponderação por TF-IDF tende a apresentar melhores resultados
- E se as palavras forem filtradas (e.g., usando k-best Selection)?
  - Podemos considerar apenas os embeddings das k-best palavras
  - Ou continuar usando os embeddings de todas as palavras do texto

Alternativa 2: classificação de sequências

- Ao invés de somar embeddings, podemos simplesmente concatená-los para uso com modelos de classificação de sequências

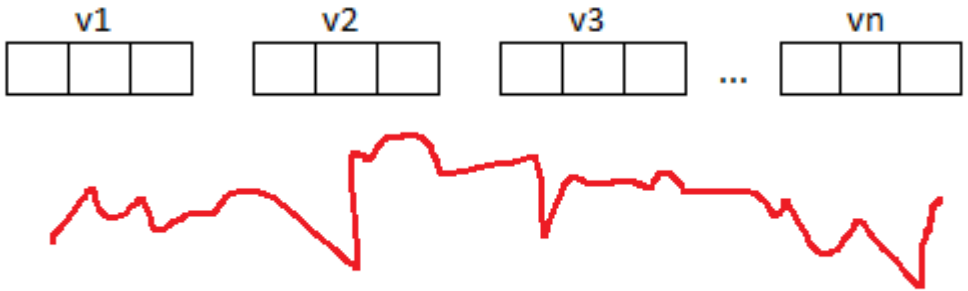
- *Embeddings* são concatenados em blocos de tamanho fixo pré-definido, completados com zero se necessário (*padding*)

- Redes neurais exploram este modelo para reconhecer padrões de texto da mesma forma que em qualquer sinal

    - Sequência de sinais = sequência de *tokens*

Textos como sequências de *tokens*

- Redes do tipo CNN e outras possuem grande capacidade de reconhecer e comparar sequências



- Resultados aproximam o estado-da-arte em classificação de textos
  - Modelos neurais em diversas bibliotecas treinam sua própria camada de *embeddings*
  - Para problemas grandes

Exercício

De volta à classificação no córpus BRmoral

- Crie modelos de classificação de gênero usando *word embeddings*.
    1. Qual o tamanho do *embeddings* ideal para este problema / córpus (50,100,300)?
    2. Qual a diferença de resultado entre pré- e auto treinado?
    3. Filtrando-se as k=9000 melhores palavras, qual a diferença entre usar *embeddings* apenas destas palavras, ou *embeddings* de todas as palavras do documento?
    4. Qual a diferença de resultado fazendo-se soma, média ou média ponderada por TF-IDF?

Configurações alternativas

Tamanho: {300,100,50}
Origem: {self, pre}
K-best: {sim, não} usando k=9000 se sim
Estratégia: {soma, média, tfidf}

3 * 2 * 2 * 3 = 36 configurações a considerar

Pergunta: qual destes 36 modelos é a configuração de melhor resultado?

E para as demais tarefas do córpus (usando k-best apropriado)?

Embeddings versus BoW

- Exemplo: classificação de gênero no córpus BRmoral
  - Modelo BoW tradicional: um vetor de 11805 dimensões
  - Com seleção de atributos k=9000: um vetor de 9000 dimensões
  - Usando embeddings de tamanho 300: um vetor de 300 dimensões
- Para problemas menores, BoW pode apresentar mesmo resultado a um custo muito menor
  - E pode ser difícil superar um bom modelo de TF-IDF com seleção de atributos

Embeddings contextuais

Embeddings contextuais

- O conceito de *word embeddings* foi inicialmente empregado na forma estática fornecida por ferramentas como word2vec, GloVe e outros.

- Nesse formato, cada palavra é associada a um vetor denso que objetiva encapsular todos os seus contextos de uso possíveis.

Embeddings contextuais (2)

- Uma mesma palavra pode entretanto ter significados muito diferentes, e é mera coincidência que estes significados tenham uma mesma representação textual.
    - e.g., papel, banco, letra, vela...

- Significados podem ser melhor capturados se cada acepção de um termo for representada por um vetor próprio, e não combinadas em um vetor único
    - *embeddings* contextuais ou dependentes de contexto

Embeddings contextuais (3)

- Modelos pré-treinados como BERT implementam *subwords*, com resultados quase que invariavelmente superiores ao uso de palavras normais.

- Diferenças entre línguas são cada vez menos importantes (ou transparentes) em modelos mais recentes
    - e.g., *embeddings* para “gato” e “cat”.

Obrigado